BombLab 实验报告

眭罗涛 25303090064

一、实验目的

本实验要求根据每关给定的 64 位 challenge，在 defuser.c 中计算随输入变化的口令，而不是提交固定密码。实验以当前个人 bomb 为分析对象，通过 nm、objdump 查看函数、汇编和只读数据，恢复验证逻辑后用 C 实现，再将求解结果交给原炸弹验证。

二、实验内容

第 1 关

通过 phase_01 将 challenge 传给 p1_compute，再比较输入和返回值。进入 p1_compute 时，rdi = C。rdx 保存原始输入；rax >> 32 得到高 32 位；rsi >> 16 得到右移 16 位后的结果。因此：

$$\text{low} = (\text{uint32_t})\text{challenge}$$
$$\text{high} = (\text{uint32_t})(\text{challenge} >> 32)$$
$$\text{middle} = (\text{uint32_t})(\text{challenge} >> 16)$$

第一次调用前，edi = high、esi = middle，所以 p1_step1(high, middle)为：

$$\text{step1} = (\text{high} + 8U * \text{middle})\text{^}0x2770U$$

第二次调用前，edi = low、esi = high，ecx 保存 step1，由 p1_step0 得：

$$\text{step0} = (3U * \text{low})\text{^}(\text{high} + 0xd44eU)$$

最后一次调用以 step0、step1 为参数，所以最终答案为：

$$\text{answer} = \text{step0}\text{^}(\text{step1} + 0x9808U)$$

```
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
Phase 1 defused.
```

第 2 关

初值部分先计算 $2 * \text{low32}(C >> 8)$，再与 0x1e 相与并加 15,而 $0x1e = 2 * 0xf$，所以等价于：

$$v0 = 15U + 2U * ((\text{uint32_t})(C >> 8)\&0xfU)$$

随后进入循环，rdx 是数组下标 i，初值为 1；ecx 是移位量，初值为 19，每轮增加 2。每轮跳回循环起点时都会重新执行

$$\text{mov rdi, r13}$$

因此每轮都是对原始 C 右移。

第 i 轮计算：

$$\text{digit} = (\text{uint32_t})(C >> (19 + 2 * (i - 1)))\&0xfU$$
$$v[i] = 3U * (v[i - 1] + (\text{uint32_t})i) - \text{digit}$$

```
第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.
```

第 3 关

首先确定两个答案的来源

第一个输入必须等于：

$$operation = C\&7$$

第二个输入是 p3_dispatch 的返回值，其运算输入为：

$$value = (uint32_t)(C >> 3)\hat{}(uint32_t)(C >> 32)$$

p3_dispatch 通过跳转表选择分支，由于函数名中的编号不等于跳转表编号，所以必须将表项地址、中间跳板和实际函数体逐一对照，恢复后的关系为：

operation	第二个整数 result
0	$5 * value + 0x36d12$
1	$(4 * value)\hat{}0x7da1$
2	$5 * value + 0x8035$
3	$(value\hat{}0xb57a) + (value >> 2)$
4	$5 * value + 0xb59b$
5	$(value\hat{}0xf378) + (value >> 1)$
6	$(2 * value)\hat{}0x5d15$

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x581ce1ff0e4ae394
Input (unsigned decimal integers):
Phase 3 defused.
```

第 4 关

由 phase_04 可得：

$$target = (uint32_t)C\&0x7fU$$

$$salt = (uint32_t)(C >> 7)\hat{}(uint32_t)(C >> 32)$$

$$answer = walk(target, 0, 127, salt)$$

target 始终在 0～127 内，因此递归最终必然到达相等基例。

递归下降首先计算中点为

$$middle = low + ((high - low) >> 1)$$

若 target == middle：

$$return(middle + 0x87U)\hat{}salt$$

若目标在左侧，则令 high = middle − 1；

若目标在右侧，则令 low = middle + 1，salt 在各层保持不变。

当前层先把 middle + salt 保存到 ebx。递归返回后，每层都执行乘 5 和异或：

//左分支

$$return(5U * walk(target, low, middle - 1U, salt)\hat{}(middle + salt))\hat{}0x0cU$$

//右分支

$$return(5U * walk(target, middle + 1U, high, salt)\hat{}(middle + salt))\hat{}0x0aU$$

因此最终值取决于完整二分搜索路径，而不只是叶节点。

```
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):
Phase 4 defused.
```

第 5 关

首先恢复结构体布局，使用数组下标代替真实地址，用整数 next 表示节点关系。起点由 C&7 选择初始值和根节点处理为：

$$\text{folded} = (\text{uint32_t})(C >> 6)\text{^}(\text{uint32_t})(C >> 32)$$

$$\text{folded} = ((\text{folded}\text{^}0x55a1U) * 3U + \text{current} -> \text{key})\text{^}\text{current} -> \text{tag}$$

根节点的 tag 同样参与折叠。随后每个后续节点执行：

$$\text{current} = \text{current} -> \text{next}$$

$$\text{folded} = (\text{folded} * 3U + \text{current} -> \text{key})\text{^}\text{current} -> \text{tag}$$

共额外前进 steps 次，最后输出 folded 和最后节点的 tag。节点表如下：

节点	next	tag	key
0	-1	0x0a21	0xcfbf
1	3	0x045f	0xca89
2	7	0x0981	0xd45d
3	10	0x0569	0x3adb
4	1	0x0204	0xa7ba
5	6	0x060e	0x5da8
6	8	0x034d	0xb8d9
7	9	0x013d	0xc7c1
8	2	0x086d	0xddd7
9	0	0x0039	0xb76e
10	5	0x0b59	0xd33b

```
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xde4431fa3c80db06
Input (unsigned decimal integers):
Phase 5 defused.
```

第 6 关

首先确定两次调用的编号，初始输入为：

$$\text{value} = (\text{uint32_t})(C >> 6)\text{^}(\text{uint32_t})(C >> 32)$$

第一次编号为 C&7。C&0x38 被直接用作函数表的字节偏移，由于每个函数指针占 8 字节，所以第二次编号等价于(C >> 3)&7。

调用顺序为：

$$\text{value} = \text{operations}[C\&7](\text{value})$$

$$\text{value}\text{^} = 0x4a22U$$

$$\text{value} = \text{operations}[(C >> 3)\&7](\text{value})$$

$$\text{answer} = \text{value} + 0x6c43U$$

7.3 分块二：恢复真实函数表

表项与函数体对应后，按表编号整理如下：

编号	运算
0	ror32(value, 2)
1	ror32(value, 2)
2	(8 * value + 0xde4c)^(7 * value)
3	(8 * value + 0xde4c)^(7 * value)

4	(value^(value >> 2)) + 0xfc28
5	(value^0x996d) + 8 * value
6	rol32(value, 5)^0x6e4e

其中 rol、ror 是循环移位，不能用普通移位替代，所以使用：

$$\text{rotate_left}(x, n) = (x << n) | (x >> (32 - n))$$

$$\text{rotate_right}(x, n) = (x >> n) | (x << (32 - n))$$

两次函数调用之间的异或和最后加法均不可省略，也不能交换顺序。

```
第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x37e9671c45376d5d
Input (unsigned decimal integers):
Phase 6 defused.
```

隐藏关

首先恢复整行输入布局，栈缓冲区起点为 rbp − 0xd0，标记检查位置为 rbp − 0xb0，相差 32 字节。小端存储下，0x4e45504f 对应内存字符串 OPEN，所以第 32～35 字节必须是 OPEN。数值答案从 rbp − 0xac 开始，相对缓冲区起点偏移 36。长度检查要求输入总长为 36～163 字节。因此可以使用：

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPENselectorvalue1value2

其中前 32 个字符为 A，三个整数从偏移 36 后开始。

记隐藏关 challenge 为 C，六关记录计算出的 context 为 K。定义：

$$a = (\text{uint32_t})C \text{^} (\text{uint32_t})(C >> 32)$$

$$b = (\text{uint32_t})K \text{^} (\text{uint32_t})(K >> 32)$$

第一个整数 selector 为

$$\text{selector} = (\text{uint32_t})K \& 3U$$

$$\text{salt} = (\text{selector} * 0xd0d1U) \text{^} (\text{uint32_t})(K >> 16)$$

由 secret_mix

$$\text{mixed} = \text{rol32}(a \text{^} b, 1)$$

$$\text{mixed} = (\text{mixed} + 0xd0d1U) \text{^} 0x2b62U$$

第二个整数 value1 为

$$\text{value1} = \text{mixed} \text{^} \text{salt}$$

随后

$$\text{extra} = b + \text{selector} + 0x2b62U + (a >> 1)$$

第二个整数 value2 为

$$\text{value2} = \text{value1} \text{^} \text{extra}$$

最终依次输出 selector、value1、value2。

```
Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

三、实验结果与总结

本实验将六关求解分别归结为函数组合、递推数列、跳转表分派、递归折叠、链表遍历和函数指针表调用，隐藏关还需要同时处理输入内存布局和数值计算。分析过程中的关键认识如下：

1.不能只看函数名称，要追踪真实参数、返回值和表项。

2.32 位截断和溢出会直接影响后续异或结果，必须保持运算宽度一致。

3.递归题既要分析向下调用，也要分析逐层返回后的处理。

四、AI 辅助说明

在递归链表和函数指针的部分使用 Kimi 辅助解释汇编，分析每次调用对应的地址，随后由本人整理求解逻辑并验证。